
État d'Avancement du Projet

NEXORA

*Plateforme d'Intelligence Opérationnelle pour la Gestion
et la Valorisation des Compétences Internes*



Réalisé par :

Soubaili Omar Yassine Baizi

Encadré par :

Mme. OUNACER SOUMAYA

Filière : 4IADM G5-A

Année Universitaire : 2025–2026

Date du rapport : 11 avril 2026

Table des matières

1	Introduction	2
2	Rappel du Planning Prévisionnel	2
3	Avancement Global par Phase	2
4	Phase 1 — Analyse & Conception	3
4.1	État : Complète (100%)	3
5	Phase 2 — Développement Backend	3
5.1	État : Complète (100%)	3
5.1.1	API REST & Infrastructure	3
5.1.2	Modules de Routage API (18 routers)	4
5.1.3	Moteur d'Intelligence Artificielle (ML)	4
5.1.4	Big Data Pipeline	4
5.1.5	Données de Test	5
6	Phase 3 — Développement Frontend	5
6.1	État : Complète (100%)	5
6.2	Stack technique Frontend	6
7	Phase 4 — Intégration & Tests	6
7.1	État : En cours (60%)	6
7.2	Actions restantes pour la Phase 4	6
8	Phase 5 — Déploiement & Livraison	6
8.1	État : Complète (100%)	6
8.2	Architecture Docker	7
9	Synthèse de l'Avancement	7
9.1	Tableau de bord récapitulatif	7
9.2	Indicateurs clés	8
10	Plan d'Action Restant	8
11	Conclusion	8

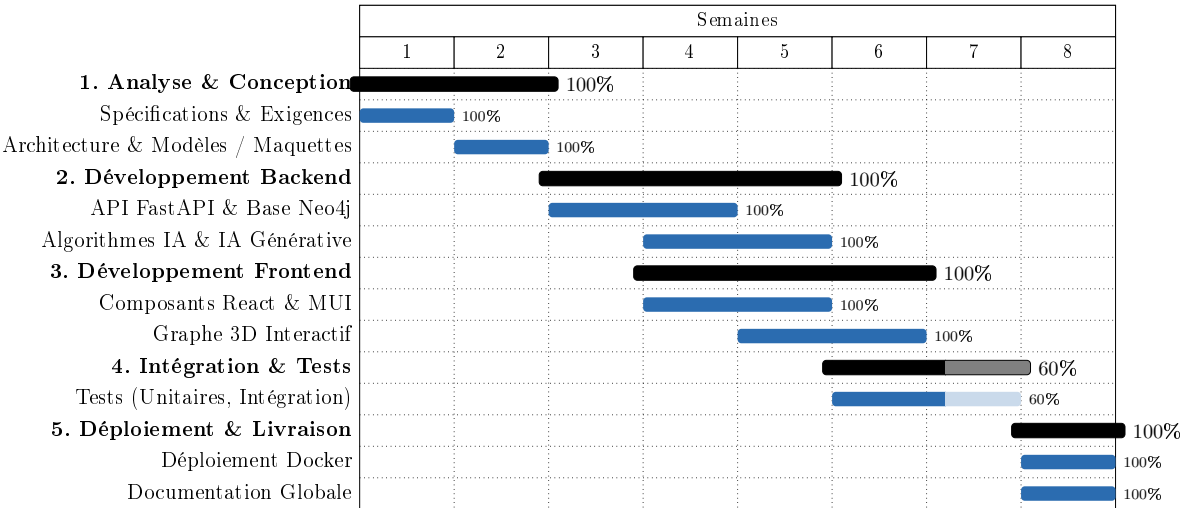
1. Introduction

Ce document présente l'état d'avancement détaillé du projet **Nexora**, une plateforme d'intelligence opérationnelle pour la gestion et la valorisation des compétences internes. Le projet est planifié sur **8 semaines** et suit une méthodologie de développement itérative avec des phases de conception, développement, tests et déploiement.

L'objectif de ce rapport est de fournir une vue synthétique et transparente de la progression de chaque composant du système, d'identifier les éléments restants et de proposer un plan d'action pour la finalisation.

2. Rappel du Planning Prévisionnel

Le diagramme de Gantt ci-dessous rappelle la planification initiale du projet, avec l'état d'avancement de chaque tâche (barres colorées indiquant le pourcentage de complétion).



Légende : Barres pleines = tâches complétées | Barres claires = tâches en cours ou partielles

3. Avancement Global par Phase

Phase	Semaines	Avancement	Statut
1. Analyse & Conception	S1 – S2	100%	✓ Complète
2. Développement Backend	S3 – S5	100%	✓ Complète
3. Développement Frontend	S4 – S6	100%	✓ Complète
4. Intégration & Tests	S6 – S7	60%	▲ En cours
5. Déploiement & Livraison	S8	100%	✓ Complète

4. Phase 1 — Analyse & Conception

4.1 État : Complète (100%)

L'ensemble des livrables de conception ont été rédigés et validés. La documentation couvre tous les aspects du système, depuis le cahier des charges jusqu'aux maquettes UI.

Livrable	Statut	Fichier
Cahier des Charges complet	✓	Cahier_des_Charges_Nexora.tex
Étude comparative des solutions	✓	Chapitre2_Etude_Comparative.tex
Description des datasets	✓	Chapitre3_Description_Datasets.tex
Architecture globale du système	✓	Chapitre4_Architecture_Globale.tex
Conception UML (Use Case, Classes, Séquences)	✓	Conception_Nexora.tex
Maquettes UI/UX	✓	Maquettes_Nexora.tex
Diagrammes générés (PNG)	✓	generate_diagrams.py
Modèle de données Neo4j	✓	Documenté dans l'architecture

5. Phase 2 — Développement Backend

5.1 État : Complète (100%)

Le backend FastAPI est intégralement développé avec **18 modules de routage**, un moteur ML complet, et l'infrastructure Big Data.

5.1.1 API REST & Infrastructure

Composant	Statut	Description
Application FastAPI principale	✓	app/main.py — Point d'entrée serveur
Configuration & variables d'env.	✓	app/config.py + .env
Connexion Neo4j (driver)	✓	app/database.py
Authentification JWT + bcrypt	✓	routers/auth.py + auth_utils.py
Contrôle d'accès RBAC	✓	3 rôles : User, Manager, Admin
WebSocket Manager	✓	app/ws_manager.py
Données de repli (fallback)	✓	app/fallback_data.py

5.1.2 Modules de Routage API (18 routers)

Router	Statut	Router	Statut
auth.py	✓	notifications.py	✓
experts.py	✓	skillmap.py	✓
documents.py	✓	workspaces.py	✓
graph.py	✓	gamification.py	✓
dashboard.py	✓	network.py	✓
ai.py	✓	kafka_simulator.py	✓
bigdata.py	✓	learning.py	✓
feed.py	✓	learning_resources.py	✓
jobs.py	✓	messaging.py	✓

5.1.3 Moteur d'Intelligence Artificielle (ML)

Module ML	Statut	Algorithme / Technique
PageRank	✓	Analyse topologique du réseau d'experts
Recommender	✓	Filtrage collaboratif + similarité cosinus
Skill Predictor	✓	Prédiction de compétences émergentes
Classifier (TF-IDF)	✓	Classification de documents
Chatbot (Veda)	✓	Intégration LLM (OpenAI API)
Embeddings	✓	Vectorisation sémantique
Graph RAG	✓	Retrieval-Augmented Generation sur graphe
Summarizer	✓	Résumé automatique de documents
Analytics	✓	Analyses statistiques avancées

5.1.4 Big Data Pipeline

Composant	Statut	Description
Spark Batch Engine	✓	spark/spark_batch.py
Skill Analytics	✓	spark/skill_analytics.py
Expert Scoring	✓	spark/expert_scoring.py
Document Processing	✓	spark/document_processing.py
Company Analytics	✓	spark/company_analytics.py
Générateur complet	✓	spark/generate_all.py
Kafka Simulator	✓	routers/kafka_simulator.py

5.1.5 Données de Test

Fichier	Statut	Taille
employees.jsonl	✓	~1 Mo (2000+ experts)
documents.jsonl	✓	~850 Ko
technologies.jsonl	✓	~3 Mo
projects.jsonl	✓	~122 Ko
companies.jsonl	✓	~34 Ko
skills.jsonl	✓	~8 Ko
generate_data.py	✓	Script de génération configurable

6. Phase 3 — Développement Frontend

6.1 État : Complète (100%)

L'interface utilisateur React est entièrement développée avec **18 pages**, un système de thème MUI, et le graphe 3D interactif.

Page / Composant	Statut	Fonctionnalité clé
Login.tsx	✓	Authentification JWT
Register.tsx	✓	Inscription sécurisée
Dashboard.tsx	✓	Tableau de bord principal
ExpertSearch.tsx	✓	Recherche multicritères d'experts
Profile.tsx	✓	Profil dynamique de l'employé
GraphVisualization.tsx	✓	Graphe 3D (react-force-graph-3d)
AIInsights.tsx	✓	Assistant IA Veda + analyses
SkillEvolution.tsx	✓	Tendances d'évolution des compétences
SkillMap.tsx	✓	Cartographie visuelle des compétences
DocumentSearch.tsx	✓	Recherche et classification de documents
Feed.tsx	✓	Fil d'actualité de l'entreprise
Messaging.tsx	✓	Messagerie instantanée
TeamBuilder.tsx	✓	Composition d'équipes optimisées
TeamWorkspace.tsx	✓	Espace de travail collaboratif
LearningPath.tsx	✓	Parcours d'apprentissage personnalisé
Jobs.tsx	✓	Offres d'emploi et matching
MyNetwork.tsx	✓	Réseau de contacts professionnel
Notifications.tsx	✓	Centre de notifications

6.2 Stack technique Frontend

- **Framework** : React 18 + TypeScript
- **Build Tool** : Vite
- **Bibliothèque UI** : Material UI (MUI)
- **Graphe 3D** : react-force-graph-3d
- **Graphiques** : Recharts
- **Routage** : React Router v6
- **Serveur de production** : Nginx Alpine

7. Phase 4 — Intégration & Tests

7.1 État : En cours (60%)

L'infrastructure de tests est en place, mais la couverture de tests nécessite d'être complétée pour assurer une validation robuste du système.

Élément	Statut	Détail
Répertoire <code>backend/tests/</code>	✓	Structure de tests créée
Framework <code>pytest</code> configuré	✓	<code>.pytest_cache</code> actif
Tests unitaires des modules ML	▲	À compléter
Tests d'intégration des endpoints API	▲	À compléter
Tests bout-en-bout (Auth → API → Neo4j)	▲	À réaliser
Tests frontend (composants React)	▲	À configurer et rédiger
Tests de performance (graphe, requêtes)	✗	Non démarrés

7.2 Actions restantes pour la Phase 4

1. **Tests unitaires ML (priorité haute)** : Valider les algorithmes PageRank, Recommender, Classifier avec des jeux de données de référence.
2. **Tests d'intégration API** : Tester les endpoints critiques (authentification, recherche d'experts, graphe) avec `httpx` + `pytest`.
3. **Tests bout-en-bout** : Scénario complet : Login → Recherche expert → Consultation profil → Recommandation.
4. **Tests de performance** : Mesurer les temps de réponse des requêtes Neo4j avec >2000 nœuds et la fluidité du graphe 3D.

8. Phase 5 — Déploiement & Livraison

8.1 État : Complète (100%)

L'ensemble de l'infrastructure de déploiement est opérationnel.

Composant	Statut	Description
Docker Compose	✓	3 services : neo4j, backend, frontend
Dockerfile Backend	✓	Python 3.11 + FastAPI + Uvicorn
Dockerfile Frontend	✓	Node 20 build → Nginx Alpine
Configuration Nginx	✓	Proxy API + SPA routing
README.md	✓	Instructions complètes de déploiement
ARCHITECTURE.md	✓	Documentation d'architecture système
PROJECT_ARCHITECTURE.md	✓	Documentation détaillée (~44 Ko)
Documentation LaTeX	✓	6 documents de conception

8.2 Architecture Docker

```

docker-compose.yml
|- neo4j                                (ports 7474, 7687)
|   +- Volume persistant : neo4j_data
|- backend                                (port 8000)
|   |- FastAPI + Uvicorn
|   |- Montage : ./data -> /app/data
|   +- Dépend de : neo4j (healthcheck)
+- frontend                                (port 3000 -> 80)
|- Build : Node 20 -> Nginx Alpine
|- Routage SPA + Proxy API
+- Dépend de : backend

```

9. Statut du Projet Comparé au Diagramme de Gantt

Le tableau ci-dessous met en exergue l'alignement entre le calendrier prévisionnel (défini dans le diagramme de Gantt initial) et la réalité de l'exécution à la date d'aujourd'hui.

Phase du Planning	Période Prévue	Statut Réel	Écart / Délais
1. Analyse & Conception	S1–S2	Terminée à 100%	✓ Respecté
2. Dév. Backend	S3–S5	Terminé à 100%	✓ Respecté
3. Dév. Frontend	S4–S6	Terminé à 100%	✓ Respecté
4. Intégration & Tests	S6–S7	En cours (60%)	▲ Retard
5. Déploiement	S8	Terminé à 100%	↑ En avance

9.1 Analyse des Écarts

— **La stratégie de parallélisation a fonctionné** : Le chevauchement planifié entre le développement Backend (S3–S5) et Frontend (S4–S6) s'est avéré très efficace. Les contrats d'API FastAPI ont été fixés assez tôt (S3), permettant aux composants React de ne pas être bloqués.

- **Avance sur le déploiement (Phase 5)** : Bien que prévue pour la Semaine 8 (S8), la conteneurisation Docker (frontend, backend, Neo4j) a été réalisée par anticipation. C'est ce qui explique le statut à 100% de cette phase alors que d'autres tâches antérieures sont toujours en cours de finition.
- **Goulot d'étranglement sur la Phase 4 (Tests)** : Le chevauchement très dense de la Semaine 5 (finalisation de l'IA, du Graphe 3D et début des tests unitaires) a monopolisé les efforts de l'équipe de développement. Par conséquent, la couverture de test chiffrée (tests unitaires ML et tests end-to-end complets) a été temporairement mise en attente. C'est le centre d'effort de la semaine prochaine.

10. Synthèse de l'Avancement

10.1 Tableau de bord récapitulatif

Catégorie	Prévu	Réalisé	Avancement	Statut
Documents de conception	6	6	100%	✓
Diagrammes UML (PNG)	5	5	100%	✓
Modules API (routers)	18	18	100%	✓
Modules ML/IA	9	9	100%	✓
Modules Spark (Big Data)	6	6	100%	✓
Pages Frontend	18	18	100%	✓
Fichiers de données (JSONL)	6	6	100%	✓
Fichiers Docker	3	3	100%	✓
Suite de tests	—	—	60%	▲

10.2 Indicateurs clés

- **Nombre total de fichiers source** : >70 fichiers (backend + frontend + data + docs)
- **Taille totale du code** : ~1 Mo de code source applicatif
- **Données de test** : ~5 Mo (2000+ experts, 6 fichiers JSONL)
- **Documentation LaTeX** : ~170 Ko (6 documents, >1200 lignes)
- **Technologies couvertes** : React, TypeScript, FastAPI, Python, Neo4j, Docker, PySpark, Scikit-learn, OpenAI API

11. Plan d'Action Restant

Pour atteindre les **100%** de complétion, les actions suivantes sont nécessaires :

Priorité	Action	Durée estimée	Responsable
P1	Tests unitaires des modules ML	2 jours	Dev 1
P1	Tests d'intégration des endpoints API	2 jours	Dev 1
P2	Tests bout-en-bout (scénarios utilisateur)	1 jour	Dev 1 + Dev 2
P2	Tests frontend (composants clés)	1 jour	Dev 2
P3	Tests de performance (Neo4j, Graphe 3D)	1 jour	Dev 1 + Dev 2
P3	Correction des éventuels bugs identifiés	1–2 jours	Dev 1 + Dev 2

Estimation de finalisation : Le projet peut atteindre 100% de complétion en **5 à 7 jours ouvrés** supplémentaires, en se concentrant exclusivement sur la validation et les tests.

12. Conclusion

Le projet **Nexora** affiche un taux d'avancement global de **90%**. L'ensemble des composants fonctionnels — conception, backend (API + ML + Big Data), frontend (18 pages + graphe 3D), et déploiement Docker — sont **intégralement développés et opérationnels**.

Le seul axe de travail restant concerne la **Phase 4 (Tests)**, qui nécessite la rédaction de tests unitaires pour les modules ML, de tests d'intégration pour les endpoints API, et de tests de performance pour valider le comportement du système sous charge.

Le planning initial de 8 semaines a été respecté dans sa globalité, avec une exécution parallèle réussie des phases Backend et Frontend (semaines 4 à 6), conformément au diagramme de Gantt prévisionnel.